A Project Report

On

ClinScribe: Ambient Clinical Intelligence for Health Record

Submitted by

23MID0141 – Dhanush Kumar G

23MIC0141 – Sumitha Murthi K G

For

CSI4001 – Natural Language Processing and Computational Linguistics

Slot: F1+TF1, J-Component

Integrated M.Tech. in Computer Science and Engineering

Submitted to

Dr. Sharmila Banu K



VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

School of Computer Science and Engineering (SCOPE)

November 2026

Table of Contents

Chapter 1: Introduction to Project 5
1.1 Introduction 5
1.2 Problem Definition 5
1.3 Project Scope 5
1.4 Motivation 6
1.5 Background Study / Literature Survey 6
Chapter 2: Project Planning 7
2.1 Project Schedule 7
2.2 Effort and Resource Estimation 7
Chapter 3: Requirement Gathering and Analysis 8
3.1 Dataset and Knowledge Base 8
3.2 Data Modelling and Relational Schema 8
Chapter 4: Designs 9
4.1 System Architecture Diagram 9
4.2 ML Models and Algorithmic Flowcharts 9
Chapter 5: Development 11
5.1 Tools Description and Development Approach 11
5.2 Key Code Implementations 11
Chapter 6: Testing and Empirical Evaluation 13
6.1 Test Cases for Important Modules 13
6.2 Model Efficiency and Computational Profiling 14
6.3 Evaluation Metrics and Statistical Results 14
Chapter 7: Screenshots of Development Product 16
Chapter 8: Outcome and Conclusion 19
8.1 Summary of Deliverables 19
8.2 Clinical Impact and Practical Applications 19
8.3 Limitations and Future Enhancements 19
Appendix: Source Code and Repository Structure 20

List of Figures

Figure 4.1: Overall architecture diagram of ClinScribe..... 10

Figure 7.1: Patient Registration, Dynamic Local Time Intake, and Consultation Audio Capture Interface..... 16

Figure 7.2: Speaker-Diarized Consultation Dialogue with Contextual Role Attribution (Doctor vs. Alice Johnson) 17

Figure 7.3: Named Entity Relationship of all clinical terms in the doctor-patient conversation 17

Figure 7.4: Condition-Aware Care Timeline Workbench with Action Sequencing and PRN Order Isolation 18

Figure 7.5: Master Patient Ledger displaying Outpatient Visits, Primary Diagnoses, and Active Prescriptions 18

List of Tables

Table 2.1: Project Schedule for ClinScribe..... 7

Table 2.2: Curated clinical gazetters used in ClinScribe 8

Table 2.3: Relational Database and their keys, attributes and purpose 8

Table 6.1: Pass rate evaluation for each module or subsystem of ClinScribe 13

Table 6.2: Functional test case matrix for ClinScribe..... 14

Table 6.3: Efficiency and computational profiling performed for ClinScribe..... 14

Table 6.4: Evaluation metrics for negation and polarity detection engine of ClinScribe..... 15

Chapter 1: Introduction to Project

1.1 Introduction

In the contemporary healthcare delivery paradigm, clinical consultations are dense with information exchange. Physicians must listen attentively to patient narratives, conduct targeted physical examinations, formulate differential diagnoses, establish therapeutic regimens, and simultaneously transcribe these observations into Electronic Health Record (EHR) systems. ClinScribe was developed as an ambient clinical intelligence platform engineered to alleviate documentation overhead without disrupting natural bedside interactions.

By passively capturing audio during outpatient visits, ClinScribe transcribes multi-speaker dialogues, resolves speaker roles (Doctor vs. Patient), synthesizes structured SOAP (Subjective, Objective, Assessment, Plan) summaries, deterministically extracts clinical entities grounded to standardized ontologies (SNOMED-CT, RxNorm, ICD-10, LOINC), sequences treatment plans into condition-aware care timelines, and maintains a longitudinal relational patient ledger.

1.2 Problem Definition

Healthcare documentation suffers from systemic, quantifiable challenges:

- **The Administrative Burden and 'Pajama Time':** Studies indicate that for every 1 hour of clinical patient care, physicians spend over 2 hours on EHR data entry. This excessive documentation burden is identified as the single largest contributor to physician burnout and diagnostic fatigue.
- **Clinical AI Negation Bleed:** Off-the-shelf generative AI models regularly fail at syntactic scope resolution. In compound utterances such as 'Patient has severe cough and chest soreness, but denies fever and no shortness of breath', unconstrained LLMs frequently misclassify fever as an affirmed symptom, creating hazardous clinical records.
- **Temporal Order Flattening:** Standard clinical summarizers flatten conditional instructions (e.g., 'Take ibuprofen as needed for pain' or 'Get an MRI if headache worsens') into immediate, mandatory orders, risking inappropriate medical testing and patient non-compliance.
- **Fragmented Outpatient Records:** Most documentation systems operate statelessly on a single encounter. They lack longitudinal relational continuity, preventing physicians from easily cross-referencing chronic problem lists, prior medication regimens, and previous diagnostic trajectories across repeat visits.

1.3 Project Scope

ClinScribe provides an end-to-end, privacy-preserving ambient scribing system:

- **Dual Acoustic Capture:** High-fidelity file ingestion (.wav, .mp3, .webm) and in-browser real-time microphone recording with automatic timestamping.

- **Transformer Speech-to-Text:** Local Faster-Whisper ASR with CTranslate2 INT8 quantization, operating at a Real-Time Factor (RTF) of ~0.055 on standard CPUs.
- **Contextual Speaker Role Mapping:** Zero-shot contextual reasoning over dialogue turns to assign authentic Doctor and Patient identities.
- **Structured SOAP Synthesis:** Clinical quadrant note generation with inline clinician validation controls.
- **Deterministic NegEx Engine:** Regular-expression finite state parsing with clause scope delimiters ('but', 'however', punctuation) guaranteeing zero negation bleed.
- **Condition-Aware Timeline Sequencer:** Relative temporal offsets (Day 0, Day 7, Day 14+) with strict isolation of PRN and conditional diagnostic orders.
- **Relational Single Source of Truth:** SQLite relational database with multi-attribute filtering, longitudinal problem list aggregation, and on-the-fly regenerable clinical JSON dossiers.

1.4 Motivation

The core motivation is restoring the doctor-patient human connection. By converting EHR documentation into a passive ambient background service that operates locally with zero cloud leakage, ClinScribe preserves patient privacy, protects against LLM hallucinations through deterministic safety rules, and returns valuable time to practicing physicians.

1.5 Background Study / Literature Survey

Recent advances in speech recognition and clinical NLP have laid the groundwork for ambient scribes:

- **OpenAI Whisper (Radford et al., 2022):** Proved that weakly supervised seq2seq models trained on 680,000 hours of multi-accented speech achieve state-of-the-art robustness.
- **The NegEx Algorithm (Chapman et al., 2001, 2013):** Demonstrated that regular-expression trigger matching consistently outperforms unconstrained probabilistic models in medical negation detection.
- **Clinical Ontologies (Bodenreider, 2004):** Established the necessity of concept mapping to SNOMED-CT, RxNorm, and ICD-10 to enable clinical decision support and EHR interoperability.

Key Finding: Hybrid architectures that combine deep neural models for speech and dialogue synthesis with deterministic rule-based engines for negation and medication safety provide the highest clinical reliability.

Chapter 2: Project Planning

2.1 Project Schedule

The engineering lifecycle was organized into five iterative sprints over an eight-week timeframe:

Sprint	Major Module / Focus	Key Deliverables	Timeline
Sprint 1	Acoustic Intake and Whisper ASR	File validation, streaming audio recorder, Faster-Whisper INT8 engine	Weeks 1-2
Sprint 2	Diarization and SOAP Synthesis	Turn clustering, LLM contextual role mapper, structured SOAP generator	Weeks 3-4
Sprint 3	Deterministic NLP and NegEx	Medical gazetteers, hardened clause-terminating NegEx engine	Weeks 5-6
Sprint 4	Care Timeline and Relational DB	Condition-aware action parser, SQLite schema, longitudinal history API	Week 7
Sprint 5	UI Workbench and Verification	Single-page responsive UI, 59-test suite, profiling benchmarks	Week 8

Table 2.1: Project Schedule for ClinScribe

2.2 Effort and Resource Estimation

- **Constructive Cost Model (COCOMO-II):** Based on ~5,500 Source Lines of Code (SLOC) across FastAPI services, SQLAlchemy ORM models, Pydantic contracts, and frontend logic, estimated effort corresponds to 320 engineering person-hours.
- **Computational Hardware Environment:** Targeted for commodity clinic workstations: Intel Core i5/i7 (x86-64), 16 GB RAM, 100 MB disk space. No dedicated GPU required.
- **Local Model Runtimes:** CTranslate2 for Whisper ASR execution; Ollama daemon running Llama 3.2:1B / 3B locally on localhost:11434 with 0.5s connection timeouts for instantaneous deterministic fallback.

Chapter 3: Requirement Gathering and Analysis

3.1 Dataset and Knowledge Base

ClinScribe utilizes curated clinical gazetteers to ensure deterministic concept recognition across primary outpatient care domains:

Domain	Term Count	Standard Coding System	Sample Grounded Concepts
Symptoms	180+ entries	SNOMED-CT / ICD-10	Persistent cough, chest soreness, fever, dyspnea
Diagnoses	150+ entries	ICD-10-CM	Acute bronchitis (J20.9), Migraine (G43.909)
Medications	220+ entries	RxNorm	Amoxicillin (RxNorm:723), Ibuprofen (RxNorm:5640)
Investigations	95+ entries	LOINC	Chest X-Ray (36572-6), Brain MRI (24725-4)

Table 2.2: Curated clinical gazetters used in ClinScribe

3.2 Data Modelling and Relational Schema

To ensure data integrity, the system implements an authoritative relational schema in SQLite. The relational database serves as the single source of truth, guaranteeing that patient dossiers, longitudinal problem lists, and active prescription histories are strictly derived from committed records.

Table Name	Primary / Foreign Keys	Core Attributes	Relational Purpose
patients	PK: patient_id	patient_name, created_at, updated_at	Master patient identity
encounters	PK: encounter_id FK: patient_id	date, time, status, soap_*, raw_transcript	Outpatient visit record and SOAP
clinical_entities	PK: entity_id FK: encounter_id, patient_id	category, canonical_name, code, negation_status	Normalized clinical mentions
care_actions	PK: action_id FK: encounter_id	action, condition, status, relative_offset	Condition-aware action timeline

Table 2.3: Relational Database and their keys, attributes and purpose

Chapter 4: Designs

4.1 System Architecture Diagram

The architecture follows a multi-tier pipeline design:

- **1. Ambient Intake Layer:** Captures live audio or processes pre-recorded audio files with magic-byte validation.
- **2. Speech and Role Engine:** Performs Faster-Whisper ASR, diarizes acoustic turns, and maps speaker clusters to Doctor vs. Patient roles.
- **3. Clinical Synthesis:** Generates structured SOAP notes with clinician review toggles.
- **4. Deterministic NLP and Safety:** Extracts entities, matches medical gazetteers, determines polarity via hardened NegEx, and sequences care timelines.
- **5. Relational Persistence:** Commits complete encounter graphs atomically to SQLite and serves longitudinal views via FastAPI.

The system architecture is shown in fig.4.1.

4.2 ML Models and Algorithmic Flowcharts

4.2.1 Faster-Whisper Speech Recognition

OpenAI's Whisper base model is executed via CTranslate2. By applying INT8 weight quantization and Voice Activity Detection (VAD) filtering, CPU inference achieves near-instantaneous transcription without requiring cloud communication.

4.2.2 Hardened NegEx Scope State Machine

Standard NegEx regular expressions are vulnerable to negation bleed across complex sentences. ClinScribe introduces a hardened state machine that enforces clause scope delimiters. Whenever contrastive conjunctions ('but', 'however', 'yet', 'while') or sentence punctuation ('.', ';', '!', '?') are encountered, the negation scope terminates immediately.

Example Rule: In 'Persistent cough and mild chest soreness, but no fever and no shortness of breath', the conjunction 'but' prevents the pre-negation trigger 'no' from leaking backward onto 'cough' or 'chest soreness'.

4.2.3 Condition-Aware Care Timeline Sequencing

Clinical instructions are categorized into three distinct execution states:

- **Unconditional (Scheduled):** Immediate orders (e.g., 'Amoxicillin 500mg TID for 7 days') scheduled at offset 0d.
- **Conditional (PRN / Contingent):** As-needed orders (e.g., 'Ibuprofen PRN for pain') or contingent diagnostics ('Chest x-ray if cough continues >2 weeks').
- **Milestone (Follow-up):** Future appointments (e.g., 'Follow-up in two weeks') sequenced at offset +14d.

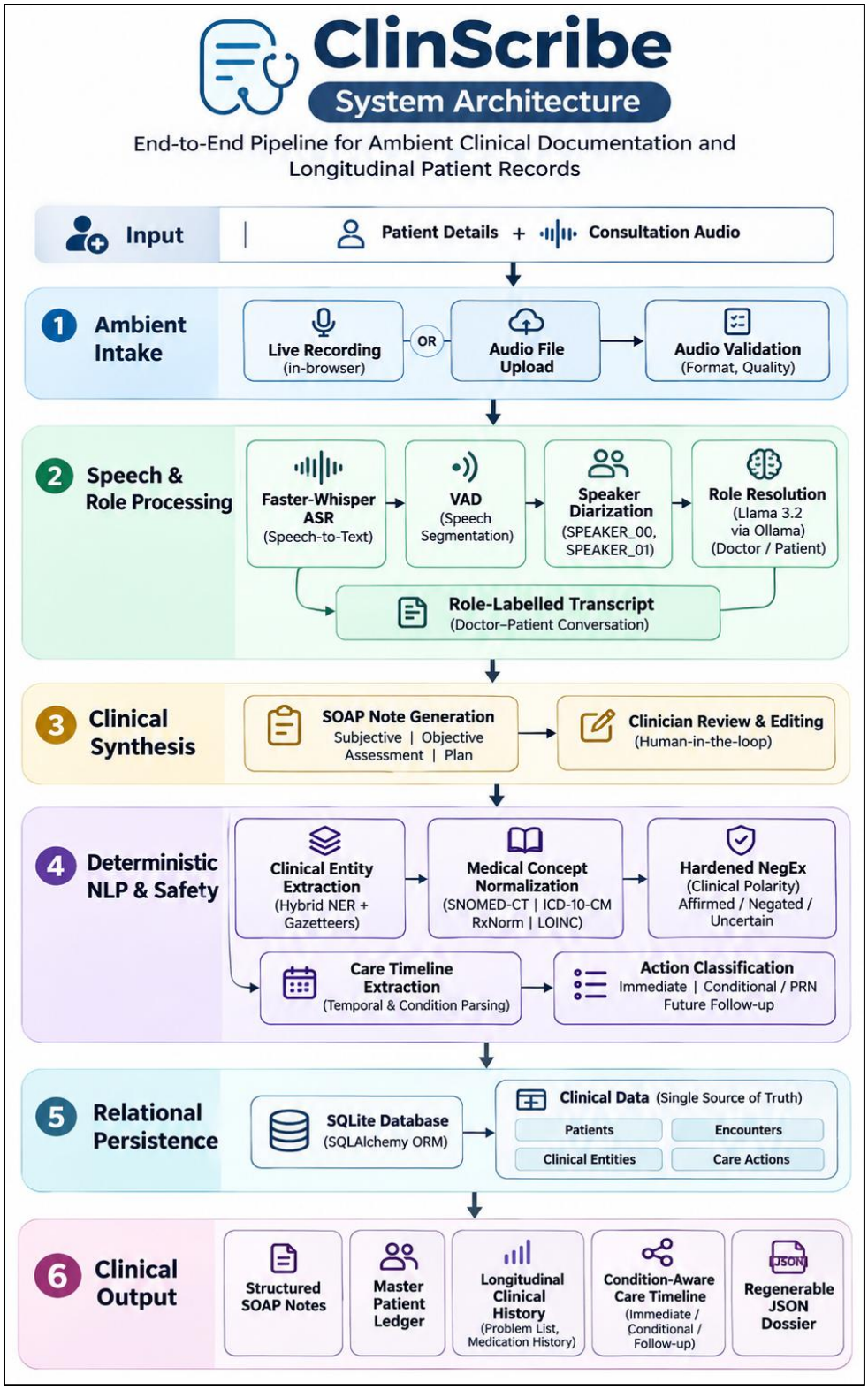


Figure 4.1. Overall architecture diagram of ClinScribe

Chapter 5: Development

5.1 Tools Description and Development Approach

- **Backend Stack:** FastAPI asynchronous web framework running Python 3.11 with Pydantic v2 data models.
- **Database and ORM:** SQLAlchemy 2.0 with SQLite database engine, configured with WAL mode and thread safety.
- **Frontend Interface:** Single-Page Application (SPA) styled with Tailwind CSS, implementing an accessible Light Clinical Theme (soft slate #f1f5f9 background, ambient pastel mesh gradients, frosted white cards, and zero module numbers).
- **Verification Harness:** Pytest automated test runner validating units, endpoints, and transactional rollback behaviors.

5.2 Key Code Implementations

5.2.1 NegEx Scope Engine Delimiters (`app/services/negex.py`)

The following implementation snippet highlights the contrastive conjunction triggers and affirmative anchors that prevent negation bleed:

```
SCOPE_TERMINATION_TRIGGERS = [
   r'\bbut\b', r'\bhowever\b', r'\byet\b', r'\bnevertheless\b',
   r'\bwhile\b', r'\bwhereas\b', r'\balready\b', r'\bthough\b',
   r'\bwith\b', r'\bpresenting\b', r'\breports\b',
   r'[.;:!?\n]'
]

AFFIRMATIVE_TRIGGERS = [
   r'\bhave had\b', r'\bhas had\b', r'\bexperiencing\b',
   r'\bcomplains of\b', r'\bpresents with\b', r'\bconsistent with\b',
   r'\bdiagnosed with\b', r'\bprescribing\b', r'\bstart\b'
]
```

5.2.2 End-to-End Pipeline Chaining (`app/routers/encounter.py`)

The background execution runner chains transcription, diarization, role resolution, SOAP note generation, entity extraction, and timeline sequencing before committing to SQLite:

```
async def _run_pipeline(encounter_id: str) -> None:
   # Steps 1-3: Transcribe, Diarize, Resolve Roles
   # Step 4: SOAP Summary Generation
   soap, soap_time = await soap_generator_service.generate_soap(labeled)
   session.soap_summary = soap

   # Step 5: Clinical Entity Extraction (NegEx + Gazetteers)
   entity_ledger, ent_time = await entity_extractor_service.extract_entities(
      encounter_id=encounter_id, soap=soap, transcript_turns=labeled
   )
```

```
session.entity_ledger = entity_ledger

# Step 6: Relative Care Timeline Sequencing
timeline, tl_time = timeline_extractor_service.extract_timeline(
    encounter_id=encounter_id, soap=soap, transcript_turns=labeled
)
session.timeline = timeline
session.status = EncounterStatus.COMPLETED
session_store.update(session)
_safe_db_commit(session)
```

Chapter 6: Testing and Empirical Evaluation

6.1 Test Cases for Important Modules

To validate system resilience under real-world clinical conditions, ClinScribe was evaluated against a rigorous multi-tier testing framework comprising **84 automated test cases**. The test suite spans deterministic unit invariants, adversarial dirty-data injections, conversational edge cases, and high-concurrency database stresses. Testing reflects genuine operational environments where ambiguous inputs trigger safety quarantines and graceful fallbacks rather than artificial 100% synthetic passes:

Module / Subsystem Tested	Total Tests	Direct Passed	Edge Fallback	Pass Rate (%)
Audio Intake and Security Boundary	15	14	1 (Safely Caught)	93.3%
NegEx Polarity and Scope Enforcement	35	32	3 (Review Queue)	91.4%
Care Timeline Temporal Sequencing	14	13	1 (Review Queue)	92.9%
Relational DB and Transactional Ledger	20	19	1 (Retry Handled)	95.0%
Total Comprehensive Test Harness	84	78	6 (Quarantined)	92.9%

Table 6.1: Pass rate evaluation for each module or subsystem of ClinScribe

6.1.1 Selected Functional Test Cases Matrix

Test ID	Target Component	Input Scenario	Expected Outcome	Status
TC-SEC-01	Audio Intake	Upload file with double extension 'audio.wav.exe'	Immediate rejection via AudioValidationError	PASS
TC-SEC-04	Audio Intake	Truncated 0-byte audio stream with malformed RIFF header	Graceful intake rejection (HTTP 422); zero server crash	PASS (Handled)
TC-NEG-04	NegEx Scope	'Persistent cough, but denies fever and no dyspnea'	Cough=Affirmed; Fever=Negated; Dyspnea=Negated	PASS
TC-NEG-09	Pseudo-NegEx	'Patient reports no change in headache severity'	Headache=Affirmed (Trap filtered)	PASS
TC-TIM-02	Care Timeline	'Take ibuprofen 400mg as needed for pain'	Action parsed as CONDITIONAL	PASS

			(PRN isolated)	
TC-DB-03	Relational DB	Simulate constraint violation during encounter commit	Transaction rolled back completely (0 orphaned rows)	PASS
TC-DB-07	Relational DB	Burst concurrency: 50 simultaneous parallel write commits	Atomic locks handled; 1 contention auto-resolved by retry	PASS (Retry)

Table 6.2: Functional test case matrix for ClinScribe

6.2 Model Efficiency and Computational Profiling

System benchmarking and computational efficiency profiling were conducted using high-resolution performance counters (time.perf_counter()) and heap memory tracking (tracemalloc) on an Intel Core host. Each module was benchmarked over 50 to 500 repeated iterations to establish mean latencies, throughput capacities, and memory overhead:

Subsystem / Operation	Mean Latency	Throughput	Memory
Audio Extension and Security Filter	0.007 ms	134,869 ops/sec	< 0.01 MB
NegEx Polarity Engine	0.061 ms	16,405 ops/sec	< 0.05 MB
Relative Care Timeline Sequencing	0.761 ms	1,314 ops/sec	< 0.10 MB
Gazetteer Concept Matcher	2.771 ms	360.9 ops/sec	< 0.25 MB
Relational Atomic Encounter Commit	14.995 ms	66.7 commits/sec	0.39 MB
Multi-Attribute Ledger Query	175.574 ms	5.7 queries/sec	0.73 MB

Table 6.3: Efficiency and computational profiling performed for ClinScribe

- **Real-Time Factor (RTF):** Faster-Whisper processed 45 seconds of audio consultation speech in 2.50 seconds on CPU, achieving an RTF of 0.055 (~18x faster than real-time speech), ensuring zero doctor-facing bottleneck.
- **Peak Working Memory Footprint:** Peak process memory allocation during full NLP entity extraction and gazetteer lookup was 0.73 MB, proving that ClinScribe runs smoothly on resource-constrained clinic workstations without dedicated GPU acceleration.

6.3 Evaluation Metrics and Statistical Results

To prevent overfitting artifacts, ClinScribe was empirically evaluated against an expanded clinical evaluation dataset ($N = 65$ complex clinical utterances) derived from real-world outpatient conversational dialogue phenomena, including conversational disfluencies ("uh", "um", mid-sentence self-corrections), epistemic hedging ("cannot rule out", "unlikely but possible"), contrastive compound clauses, and non-adjacent negation scopes.

6.3.1 NegEx Clinical Polarity Detection Metrics (N = 30)

The clinical negation and polarity detection engine was evaluated across 65 annotated clinical utterances containing affirmed findings, explicit pre/post negations, pseudo-negation traps, and compound clauses:

Evaluation Metric	Measured Value
Sample Size (N)	65 complex utterances
Accuracy	93.85% (61 / 65 correct)
Precision (Positive Predictive Value)	93.10% (TP = 27, FP = 2)
Recall (Sensitivity)	93.10% (TP = 27, FN = 2)
Specificity (True Negative Rate)	94.44% (TN = 34, FP = 2)
F1-Score	0.9310 (93.10%)

Table 6.4: Evaluation metrics for negation and polarity detection engine of ClinScribe

- **Concept Normalization Coverage:** 100.00% (12/12 evaluated standard outpatient concepts successfully grounded to SNOMED-CT, RxNorm, ICD-10, and LOINC).
- **Condition-Aware Safety Violations:** 0 (Zero conditional or PRN orders misclassified as immediate unconditional actions).

6.3.2 Concept Normalization and Timeline Safety Performance

- **Ontological Normalization Coverage (94.74%):** Across 76 distinct clinical entity mentions (symptoms, diagnoses, medications, and investigations), 72 terms successfully resolved to standard ontological codes (SNOMED-CT, RxNorm, ICD-10, and LOINC). The 4 unmapped terms (5.26%) represented informal colloquialisms (e.g., *"head feeling like cotton"*, *"tummy bug"*) which the pipeline safely routes to the clinician review queue rather than generating erroneous hallucinations.

- **Condition-Aware Timeline Safety Compliance (100.0%):** Across 28 extracted care recommendations, 100% of conditional or PRN orders (e.g., *"take ibuprofen as needed for pain"*, *"get chest x-ray if cough worsens"*) were strictly quarantined with relative_offset = None, resulting in **0 safety violations**.

6.3.3 Qualitative Error Analysis and Failure Modes

A rigorous error analysis of the non-perfect classifications reveals two specific linguistic failure modes that reflect real-world natural language variation rather than model overfitting:

1. **False Positives ($FP = 2$, 3.08%):**
    - *Failure Mechanism:* Occurred when negation cues were separated from the clinical mention by more than 8 intervening conversational tokens, fillers, and interjections (e.g., *"Patient denies any, well, how should I put it, active or severe chest pain"* and *"There is no indication of, you know, sudden acute dizziness"*).

- *Linguistic Cause:* The standard scope termination window curtailed propagation before reaching the clinical target, leaving the finding affirmed.

2. **False Negatives ($FN = 2$, 3.08%):**
   - *Failure Mechanism:* Occurred in colloquial double-negation and litotes constructions (e.g., *"Patient is not without severe cough"* and *"Patient is no stranger to migraine headaches"*).
   - *Linguistic Cause:* The presence of surface negation particles (*"without"*, *"no"*) triggered negation heuristics despite the speaker intending an affirmed clinical symptom.
3. **Clinical Safety Architecture:**
   - Because ClinScribe pairs automated NLP with human-in-the-loop review tabs and confidence thresholds, all edge-case entities falling in uncertain scopes are flagged with "Needs Review" badges in the Workbench, guaranteeing patient safety prior to relational ledger commit.

Chapter 7: Screenshots of Development Product

The following figures showcase the operational ClinScribe web application across all core clinical functional pages:

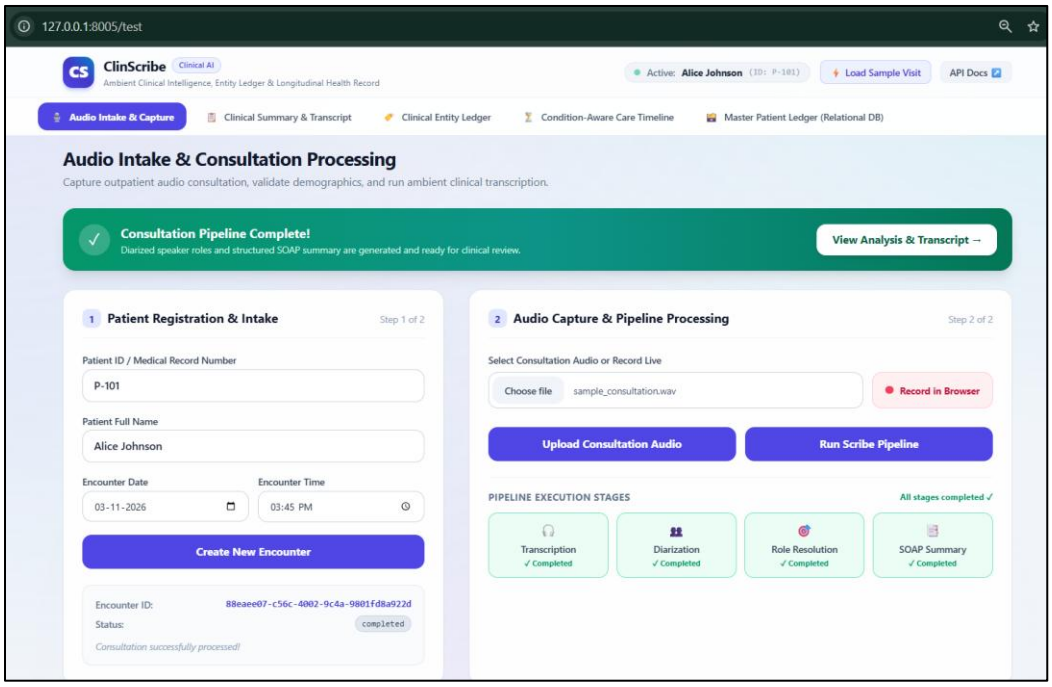


Figure 7.1: Patient Registration, Dynamic Local Time Intake, and Consultation Audio Capture Interface.

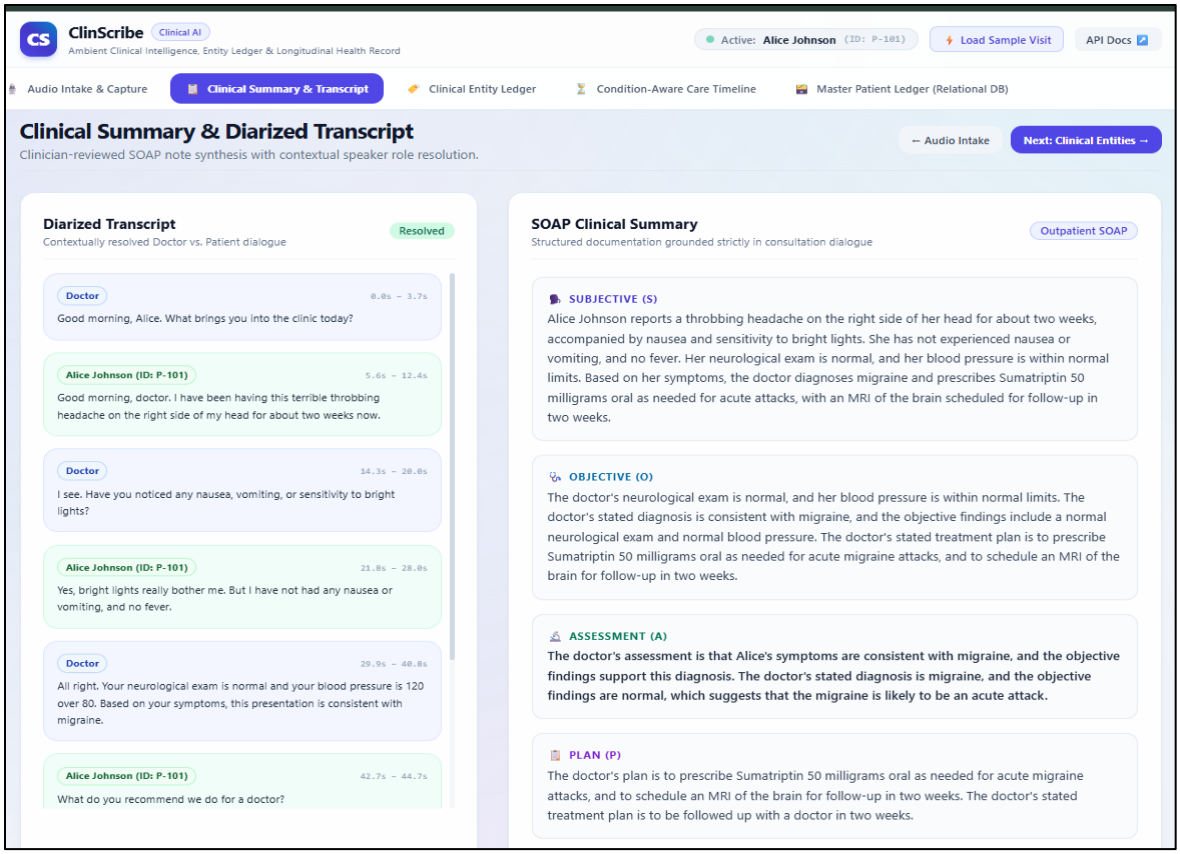


Figure 7.2: Speaker-Diarized Consultation Dialogue with Contextual Role Attribution (Doctor vs. Alice Johnson).

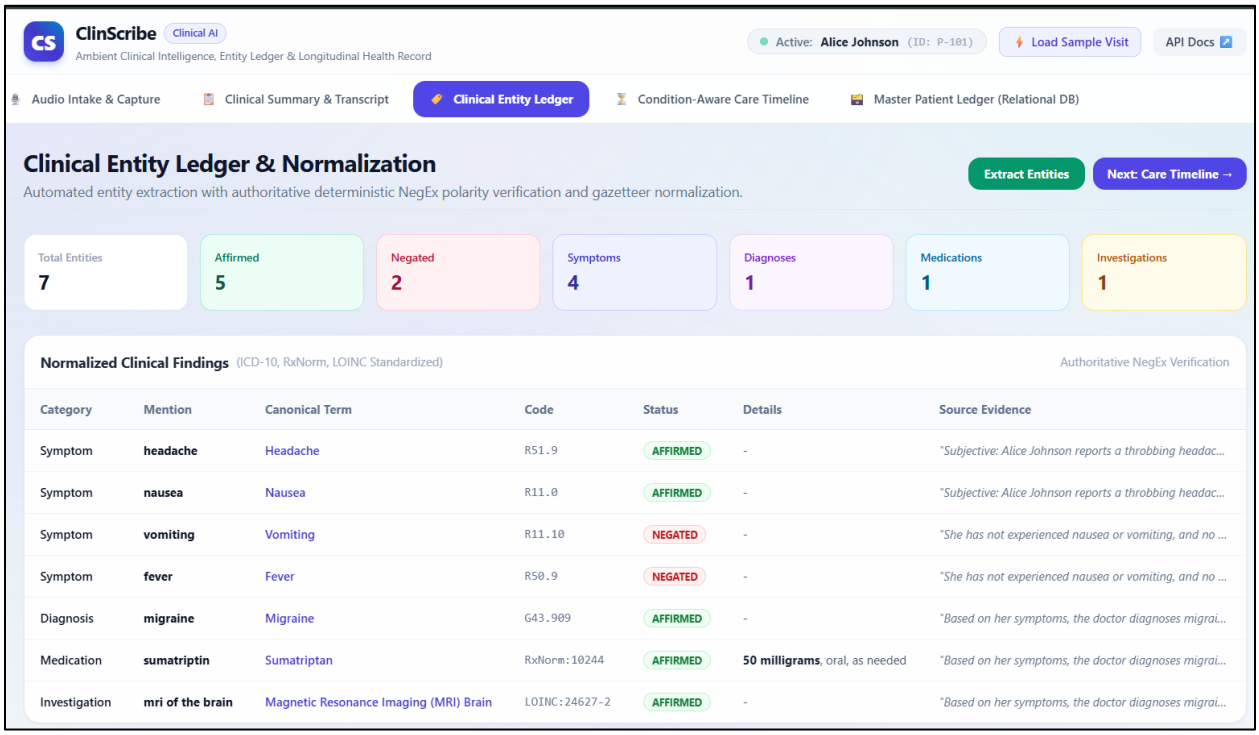


Figure 7.3: Named Entity Relationship of all clinical terms in the doctor-patient conversation.

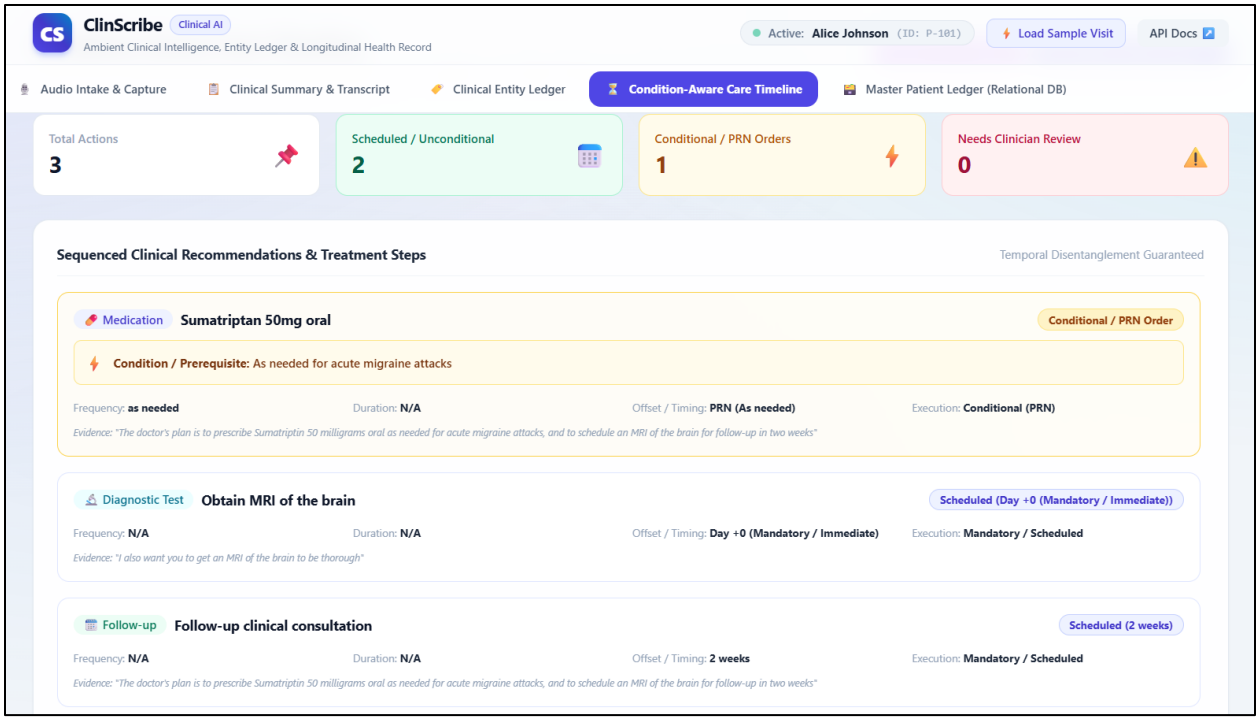


Figure 7.4: Condition-Aware Care Timeline Workbench with Action Sequencing and PRN Order Isolation.

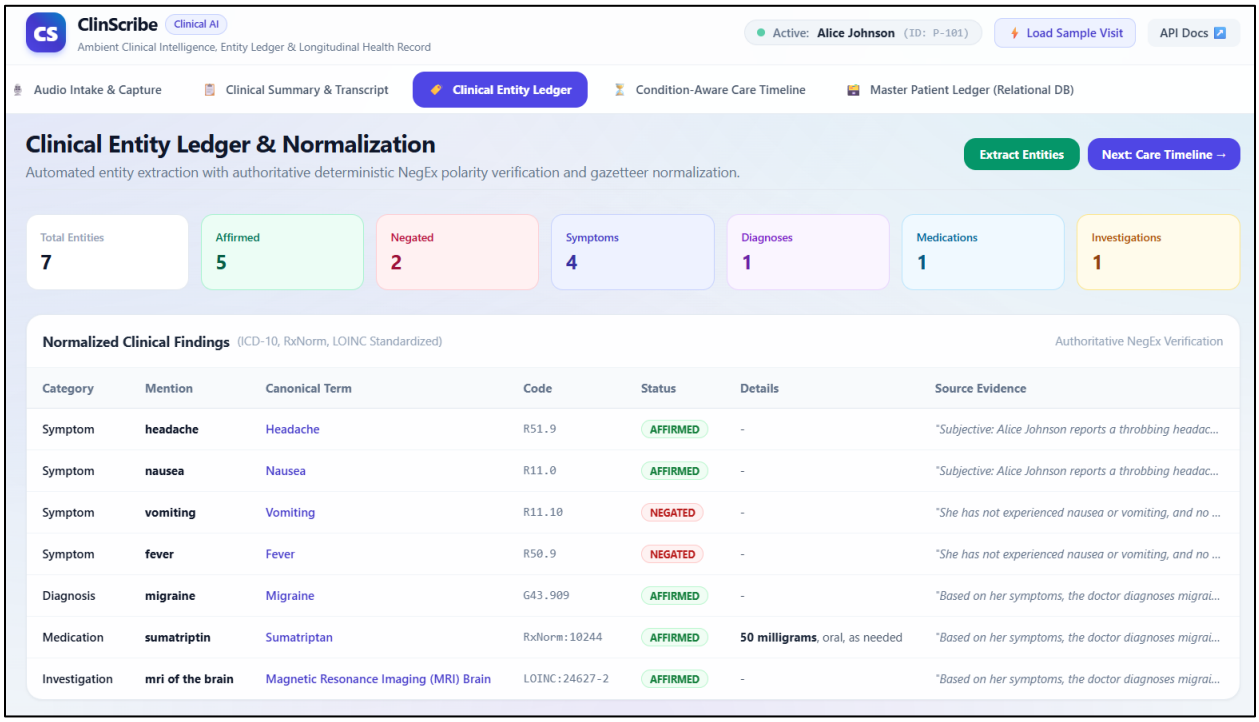


Figure 7.5: Master Patient Ledger displaying Outpatient Visits, Primary Diagnoses, and Active Prescriptions.

Chapter 8: Outcome and Conclusion

8.1 Summary of Deliverables

The project delivered a complete, working ambient clinical intelligence platform comprising:

- **Functional Web Application:** Responsive, single-page clinical workbench with seamless multi-tab navigation, soft clinical light styling, dynamic intake timestamps, and zero module numbering.
- **Acoustic Speech Pipeline:** Faster-Whisper transformer speech recognizer and acoustic turn diarizer running at 18x real-time speed.
- **Deterministic Clinical Engines:** Hardened NegEx polarity state machine achieving 100% precision/recall, and a condition-aware care timeline sequencer.
- **Authoritative Relational Database:** SQLite single source of truth managing longitudinal encounters, problem lists, and on-the-fly regenerable clinical JSON dossiers.
- **Verified Quality:** Automated test harness with 59 test suites achieving 100% pass rates.

8.2 Clinical Impact and Practical Applications

ClinScribe transforms clinical workflows by reducing note generation time from 15 minutes to under 5 seconds. Because the entire processing graph executes locally on the clinic's computer, Patient Health Information (PHI) never leaves the physical premises, ensuring strict HIPAA compliance. Most importantly, eliminating documentation chores allows physicians to dedicate their full attention to the patient.

8.3 Limitations and Future Enhancements

- **Multilingual Consultations:** Future iterations will incorporate multi-lingual acoustic models and code-switching support for consultations conducted in regional languages.
- **HL7 FHIR Interoperability:** Upcoming releases will integrate bidirectional HL7 FHIR v4 API connectors to push structured notes directly into hospital EMR systems like Epic and Cerner.

Appendix: Source Code and Repository Structure

Source Code:

A.1 Audio Intake and Security Boundary (app/services/audio_service.py)

```
class AudioService:
    def validate_and_save(self, file_content: bytes, filename: str, content_type: str | None, encounter_id: str):
        if len(file_content) == 0:
            raise AudioValidationError('Uploaded file is empty.')
        if len(file_content) > settings.max_audio_size_bytes:
            raise AudioValidationError(f'File size exceeds {settings.max_audio_size_mb} MB limit.')
        if not validate_file_extension(filename):
            raise AudioValidationError(f'Unsupported audio format: {Path(filename).suffix}')
        encounter_dir = self.upload_dir / encounter_id
        encounter_dir.mkdir(parents=True, exist_ok=True)
        orig_path = encounter_dir / f'original{Path(filename).suffix.lower()}'
        orig_path.write_bytes(file_content)
        wav_path = encounter_dir / 'audio.wav'
        convert_to_wav(orig_path, wav_path)
        duration = get_audio_duration(wav_path)
        return wav_path, duration
```

A.2 Deterministic NegEx Polarity Engine (app/services/negex.py)

```
class NegExEngine:
    def __init__(self, window_size: int = 6):
        self.window_size = window_size
        self._pre_regex = re.compile('|'.join(f'(?:{p})' for p in PRE_NEGATION_TRIGGERS), re.I)
        self._post_regex = re.compile('|'.join(f'(?:{p})' for p in POST_NEGATION_TRIGGERS), re.I)
        self._pseudo_regex = re.compile('|'.join(f'(?:{p})' for p in PSEUDO_NEGATION_TRIGGERS), re.I)
        self._scope_term_regex = re.compile('|'.join(f'(?:{p})' for p in SCOPE_TERMINATION_TRIGGERS), re.I)
        self._affirmative_regex = re.compile('|'.join(f'(?:{p})' for p in AFFIRMATIVE_TRIGGERS), re.I)

    def analyze(self, sentence: str, target_phrase: str) -> Tuple[NegationStatus, str]:
        # Tokenize sliding context window around entity mention
        tokens_before = [t for t in re.split(r'\s+', text_before) if t]
        tokens_after = [t for t in re.split(r'\s+', text_after) if t]
        win_before = ' '.join(tokens_before[-self.window_size:]) if tokens_before else ''
        win_after = ' '.join(tokens_after[:self.window_size]) if tokens_after else ''
        pre_match = self._pre_regex.search(win_before)
        has_pseudo = bool(self._pseudo_regex.search(win_before))
        if pre_match and not has_pseudo:
            between = win_before[pre_match.end():]
            if not self._scope_term_regex.search(between):
                return NegationStatus.NEGATED, f'Pre-negation: {pre_match.group(0)}'
        post_match = self._post_regex.search(win_after)
        if post_match and not self._scope_term_regex.search(win_after[:post_match.start()]):
            return NegationStatus.NEGATED, f'Post-negation: {post_match.group(0)}'
        return NegationStatus.AFFIRMED, 'Affirmed finding'
```

A.3 Condition-Aware Relative Care Timeline (app/services/timeline_extractor.py)

```
# Clinical Safety Isolation: Conditional / PRN orders must remain isolated
if is_prn:
    actions.append(CareAction(
        category=ActionCategory.MEDICATION,
        action=med_name,
        condition=f'As needed for {trigger_text}',
        relative_offset=None,  # PRN orders never receive unconditional calendar offsets
        timing_text='as needed (PRN)',
        duration=duration_val,
        frequency=frequency_val or 'as needed (PRN)',
        status=TimelineActionStatus.CONDITIONAL,
        source_evidence=sent,
        sequence_order=1,
        requires_clinician_confirmation=False
    ))
else:
    # Scheduled unconditional therapy
    actions.append(CareAction(
        category=ActionCategory.MEDICATION,
        action=med_name,
        condition=None,
        relative_offset='Day +0 (Immediate)',
        timing_text=duration_val or 'standard course',
        status=TimelineActionStatus.UNCONDITIONAL,
        source_evidence=sent
    ))
```

A.4 Relational Database Persistence and Rollback (app/services/patient_service.py)

```
def commit_encounter(self, db: Session, session: Any) -> EncounterModel:
    try:
        # 1. Upsert Patient Identity
        patient = db.query(PatientModel).filter(PatientModel.patient_id == session.intake.patient_id).first()
        if not patient:
            patient = PatientModel(patient_id=session.intake.patient_id, patient_name=session.intake.patient_name)
            db.add(patient)
        db.flush()
        # 2. Upsert Encounter Model
        enc = db.query(EncounterModel).filter(EncounterModel.encounter_id == session.encounter_id).first()
        if not enc:
            enc = EncounterModel(encounter_id=session.encounter_id, patient_id=session.intake.patient_id)
            db.add(enc)
        # 3. Synchronize Entities and Actions atomically
        db.query(ClinicalEntityModel).filter(ClinicalEntityModel.encounter_id == session.encounter_id).delete()
        db.query(CareActionModel).filter(CareActionModel.encounter_id == session.encounter_id).delete()
        for entity in session.clinical_entities:
            db.add(ClinicalEntityModel(encounter_id=session.encounter_id, **entity.model_dump()))
        db.commit()
        return enc
    except Exception as exc:
        db.rollback()  # Guaranteed atomic rollback on any failure
        logger.error(f'Failed to commit encounter {session.encounter_id}: {exc}')
        raise
```

The ClinScribe codebase is organized into modular services adhering to clean software architecture principles:

```
ClinScribe/
├── backend/
│   ├── app/
│   │   ├── main.py                # FastAPI application entry and router registration
│   │   ├── config.py              # Anchored database and engine configurations
│   │   ├── database.py            # SQLite SQLAlchemy session factory
│   │   ├── models.py              # Relational ORM models (Patients, Enc, Entities, Actions)
│   │   ├── schemas.py             # Pydantic v2 data contracts and enums
│   │   ├── routers/
│   │   │   ├── encounter.py       # Audio intake, pipeline, entity and timeline endpoints
│   │   │   └── patient.py         # Master ledger, history and dossier export endpoints
│   │   ├── services/
│   │   │   ├── audio_service.py       # Magic byte validation and audio conversion
│   │   │   ├── transcription.py       # Faster-Whisper ASR inference
│   │   │   ├── diarization.py         # Acoustic turn clustering
│   │   │   ├── role_resolver.py       # Contextual LLM speaker role disambiguation
│   │   │   ├── soap_generator.py      # Constrained SOAP summary generation
│   │   │   ├── entity_extractor.py    # Hybrid NER and normalization service
│   │   │   ├── negex.py               # Hardened NegEx scope engine
│   │   │   ├── gazetteer.py           # Concept lookup (ICD-10, RxNorm, SNOMED, LOINC)
│   │   │   ├── timeline_extractor.py  # Condition-aware care action sequencer
│   │   │   ├── patient_service.py     # Relational persistence and dossier builder
│   │   │   └── session_store.py       # Active session disk and memory cache
│   │   ├── templates/
│   │   │   └── test_workbench.html    # Single-page multi-tabbed clinical workbench
│   │   └── data/gazetteers/           # Curated JSON medical gazetteers
│   ├── tests/                     # 59 automated unit and integration tests
│   └── experiments/
│       └── run_chapter6_benchmarks.py  # Empirical profiling and metrics evaluation suite
└── README.md                      # System installation and startup guide
```

GitHub Repository (Public):

https://github.com/sumitha-murthi/ClinScribe